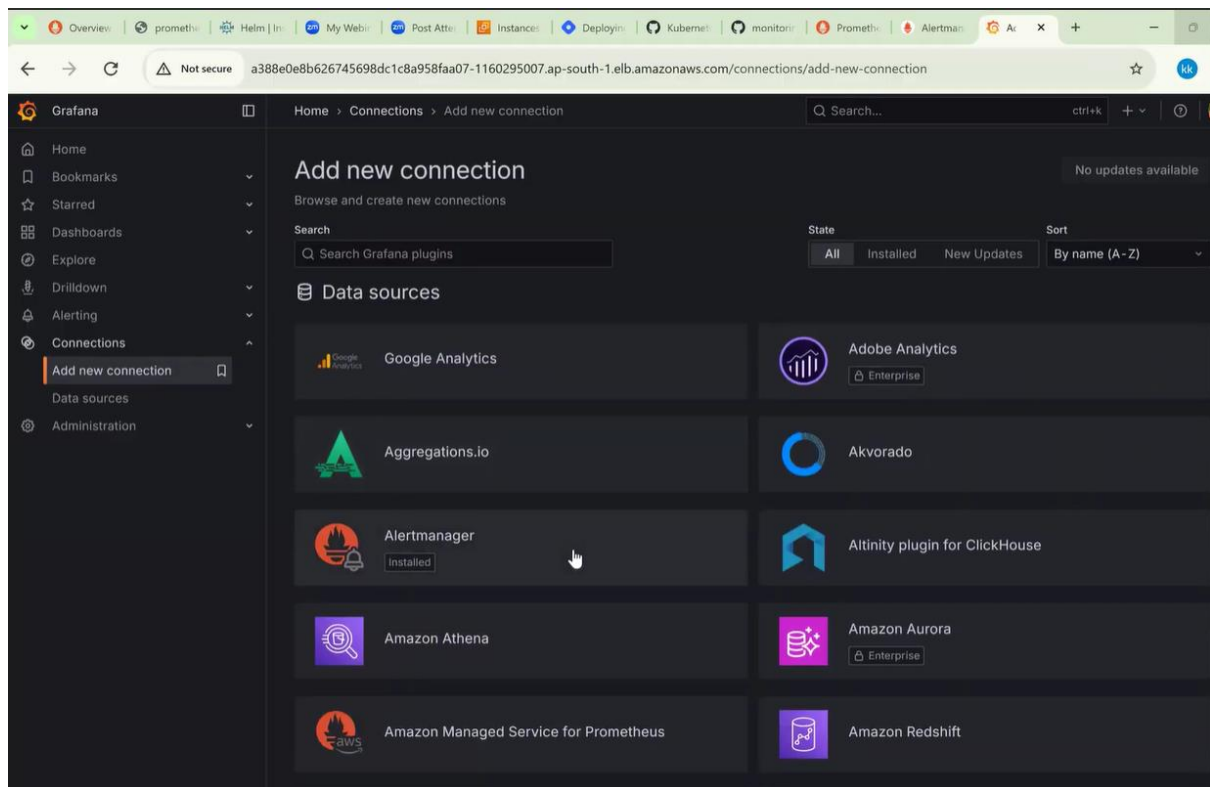
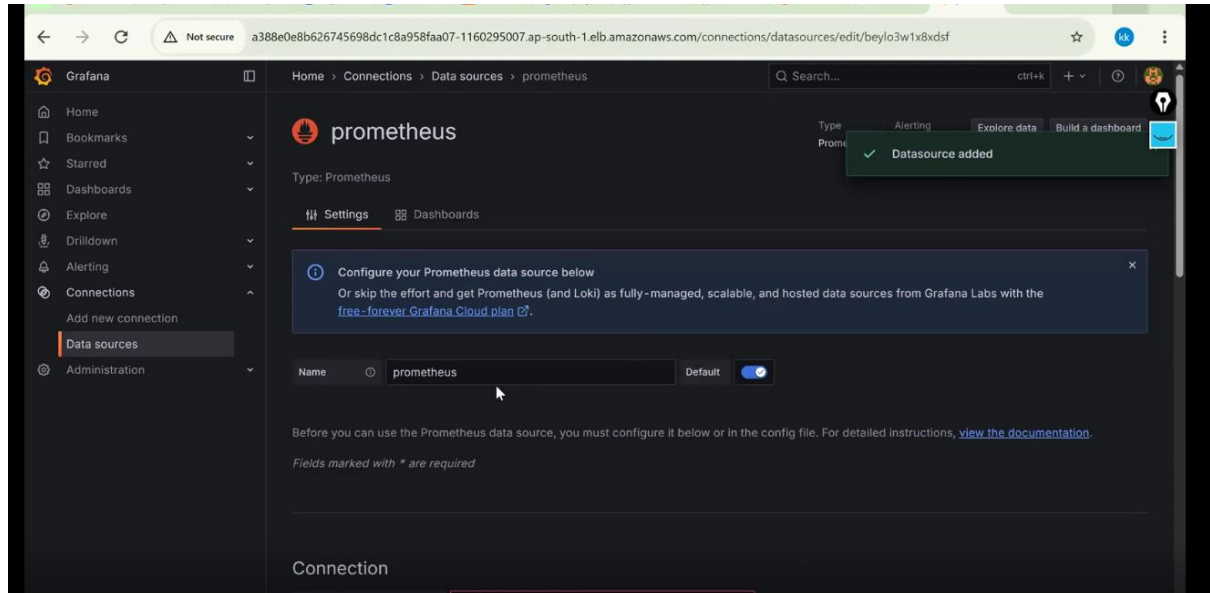


How prometheus data connectd to Grafana Dashboard

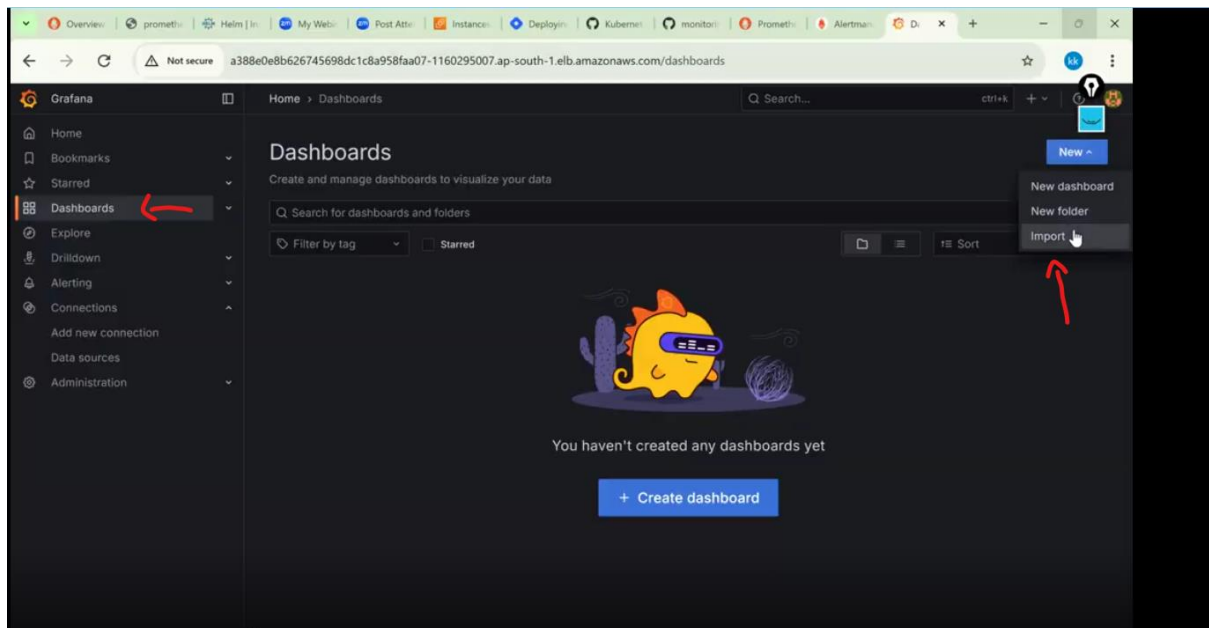


In Grafana Dashboard-----go to connections-----click---Add new connection



step-1: In Grafana Dashboard----go to **Datasource** & search for Prometheus

Click---save & next-----so connection is established for---Promethus server & Grafana



Step-2: Create Dashboard in Grafana:

Click—Dashboard ---click new---select any one of them----(New dashboard, New folder, Import)

📊 Service availability (Success rate %)

(

$$\frac{\text{sum}(\text{rate}(\text{http_requests_total}\{\text{status}=\sim"2..\"}[5\text{m}]))}{\text{sum}(\text{rate}(\text{http_requests_total}[5\text{m}]))}$$

) * 100

🔗 **Answers:** Are users getting successful responses?

📊 Error rate (5xx only)

$$\text{sum}(\text{rate}(\text{http_requests_total}\{\text{status}=\sim"5..\"}[5\text{m}]))$$

🔗 **Answers:** Are users seeing server errors?

📊 P95 latency (user-visible latency)

histogram_quantile(
 0.95,

$$\text{sum}(\text{rate}(\text{http_request_duration_seconds_bucket}[5\text{m}])) \text{ by } (\text{le})$$

)

🔗 **Answers:** Is the service slow for users?

4 Traffic (requests per second)

```
sum(rate(http_requests_total[5m]))
```

🔗 **Answers:** Is traffic normal or spiking?

5 Error budget burn rate

1 -

(

```
sum(rate(http_requests_total{status=~"2.."}[5m]))
```

/

```
sum(rate(http_requests_total[5m]))
```

)

🔗 **Answers:** How fast are we burning reliability?

6 Pod restart rate

```
sum(rate(kube_pod_container_status_restarts_total[5m]))
```

🔗 **Answers:** Is the platform unstable?

7 CPU saturation (usage vs limit)

```
sum(rate(container_cpu_usage_seconds_total[5m]))
```

/

```
sum(kube_pod_container_resource_limits_cpu_cores)
```

🔗 **Answers:** Are we hitting CPU limits?

8 Memory saturation

```
sum(container_memory_working_set_bytes)
```

/

```
sum(kube_pod_container_resource_limits_memory_bytes)
```

📌 **Answers:** Risk of OOM kills?

🔍 Instance availability (up)

```
sum(up == 1)
```

📌 **Answers:** How many targets are actually reachable?

10 Dependency latency (downstream service)

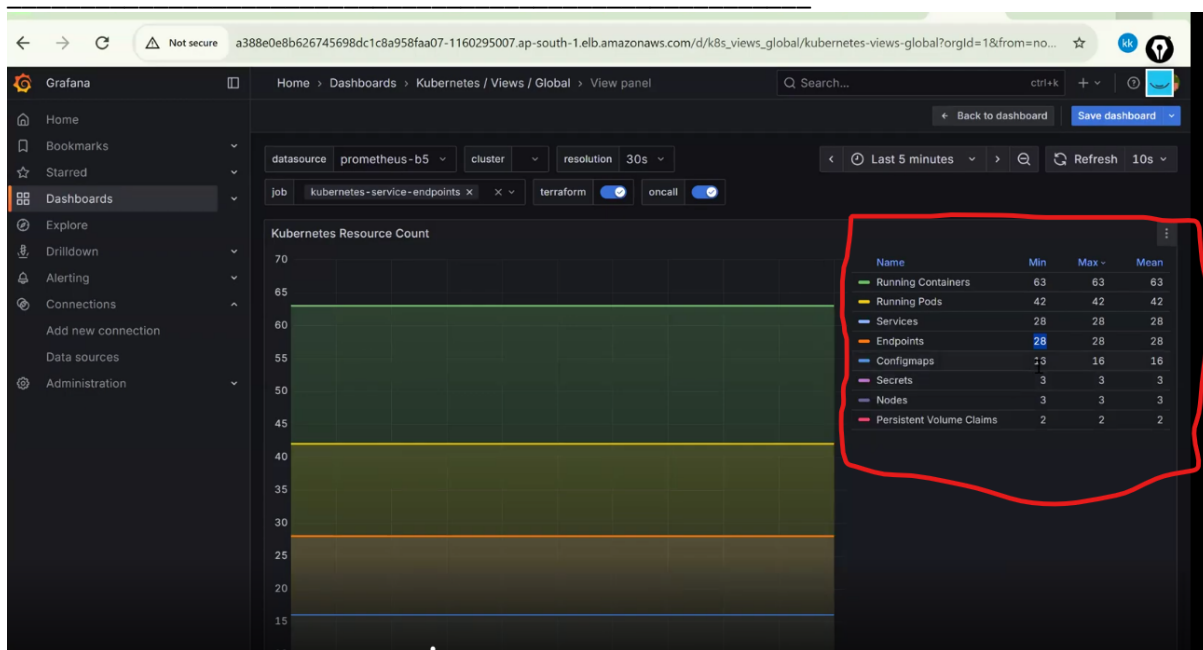
```
histogram_quantile(
```

```
0.95,
```

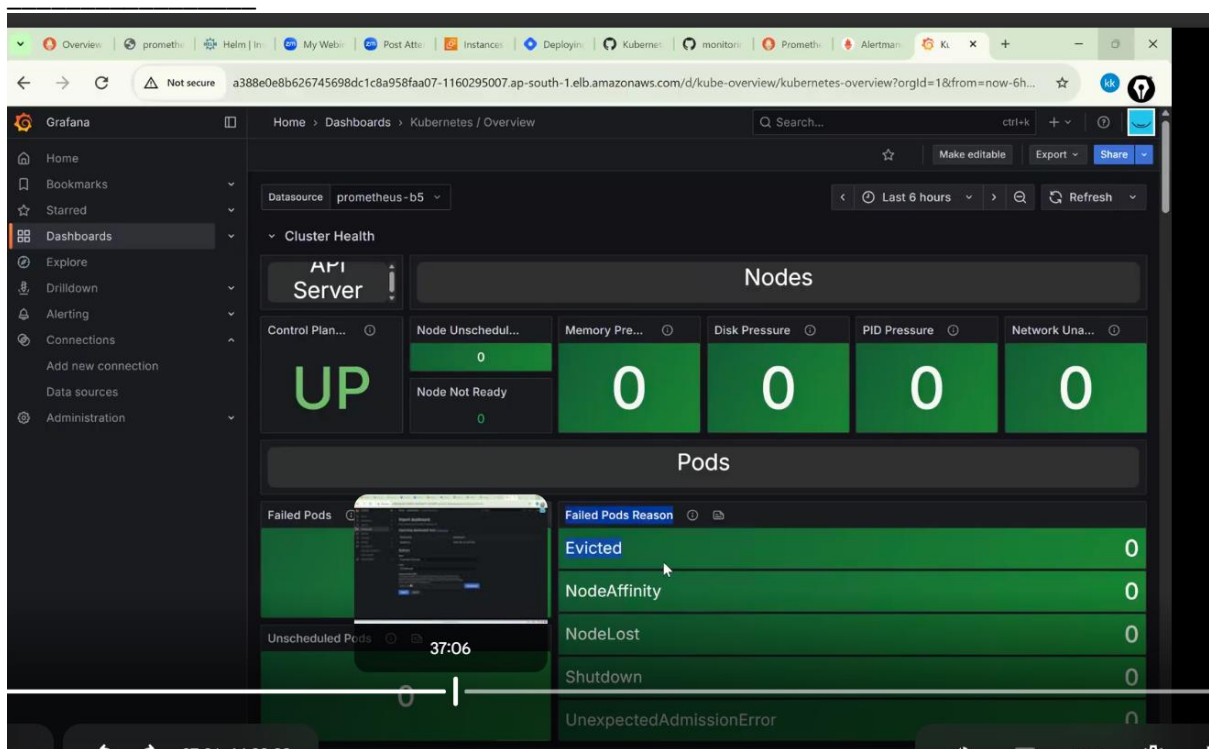
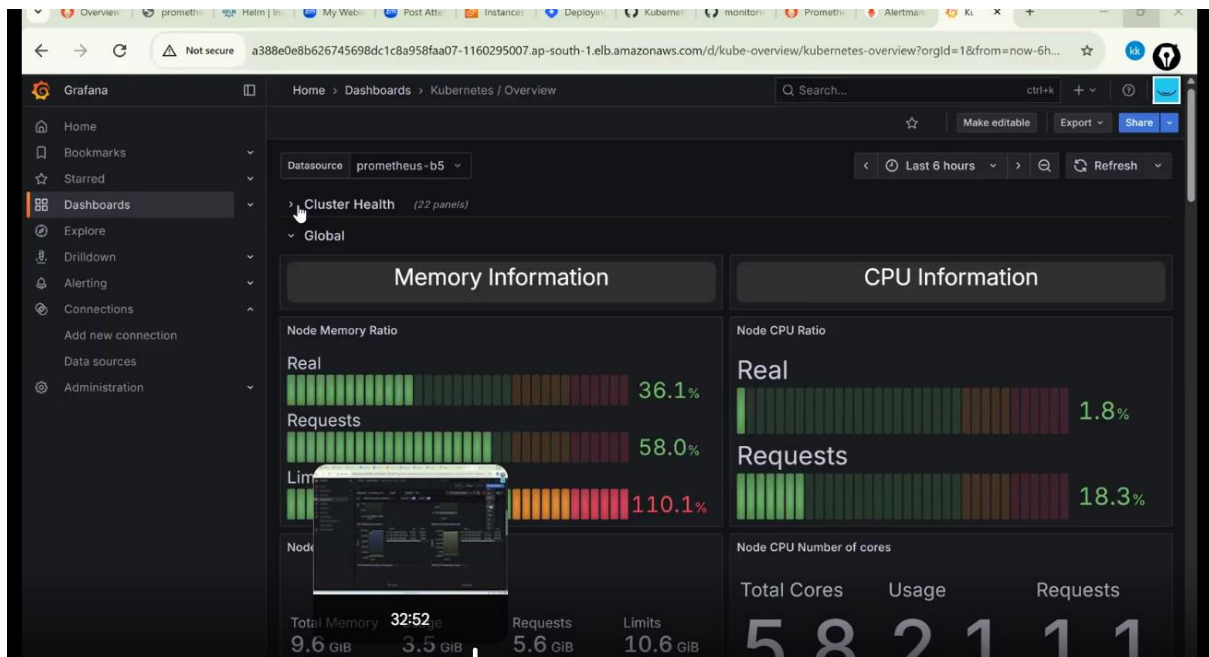
```
sum(rate(http_client_request_duration_seconds_bucket[5m])) by (le)
```

```
)
```

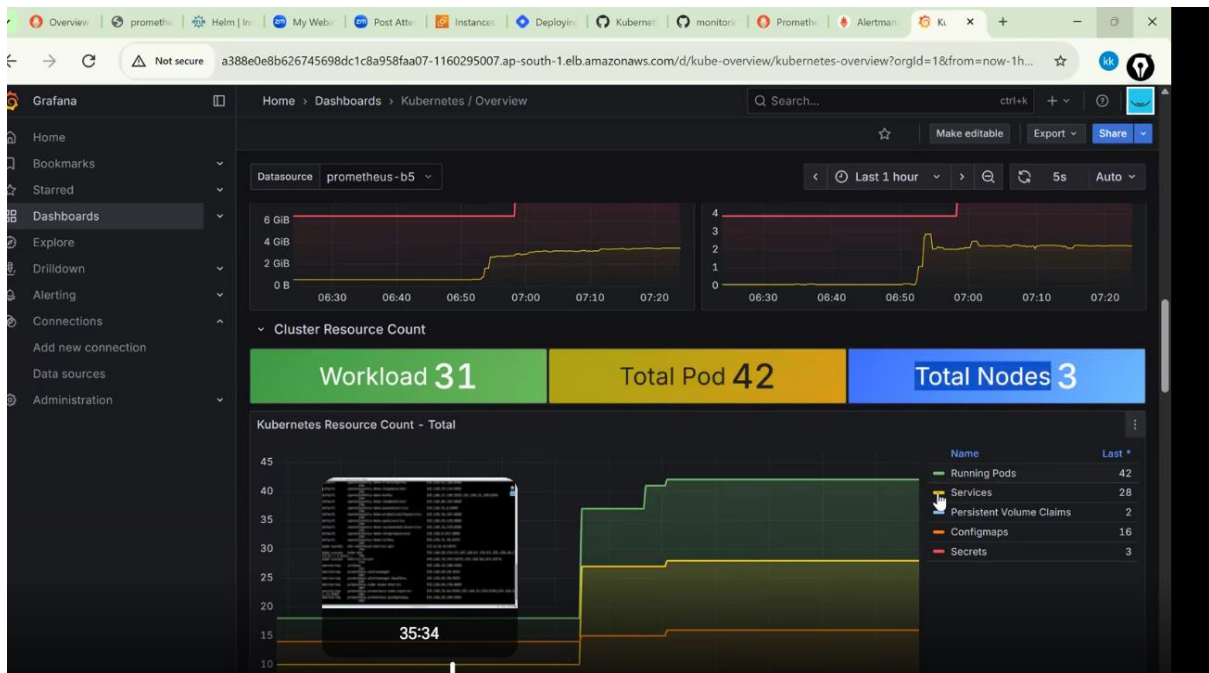
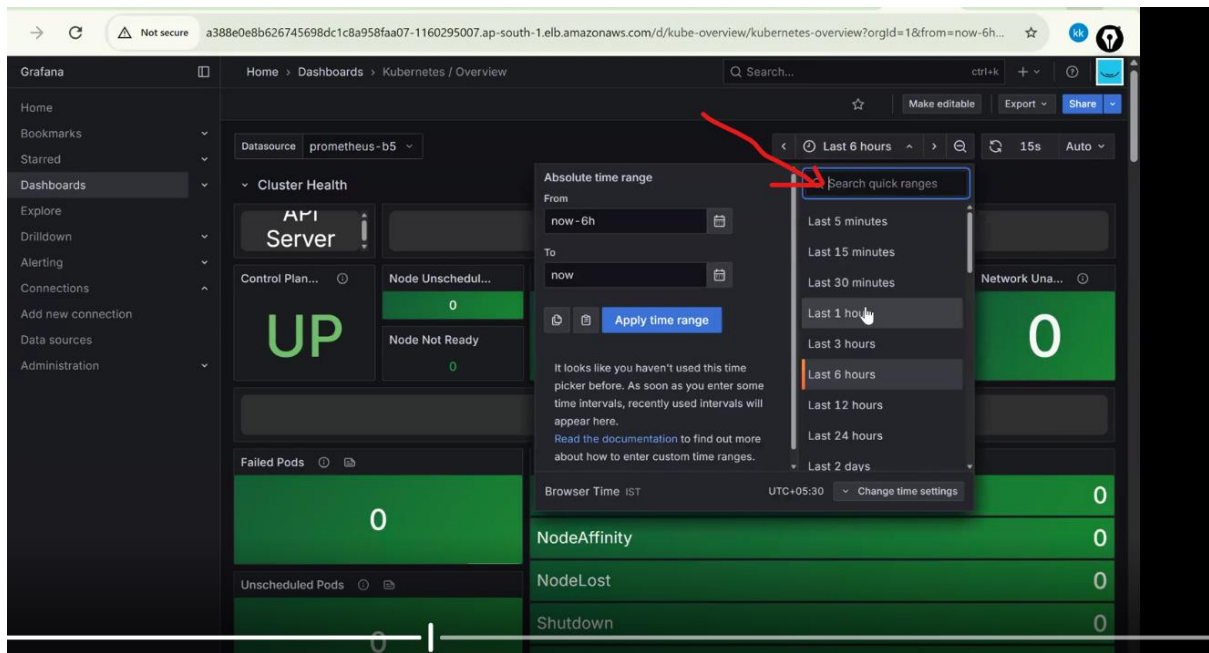
📌 **Answers:** Is a downstream dependency slowing us?

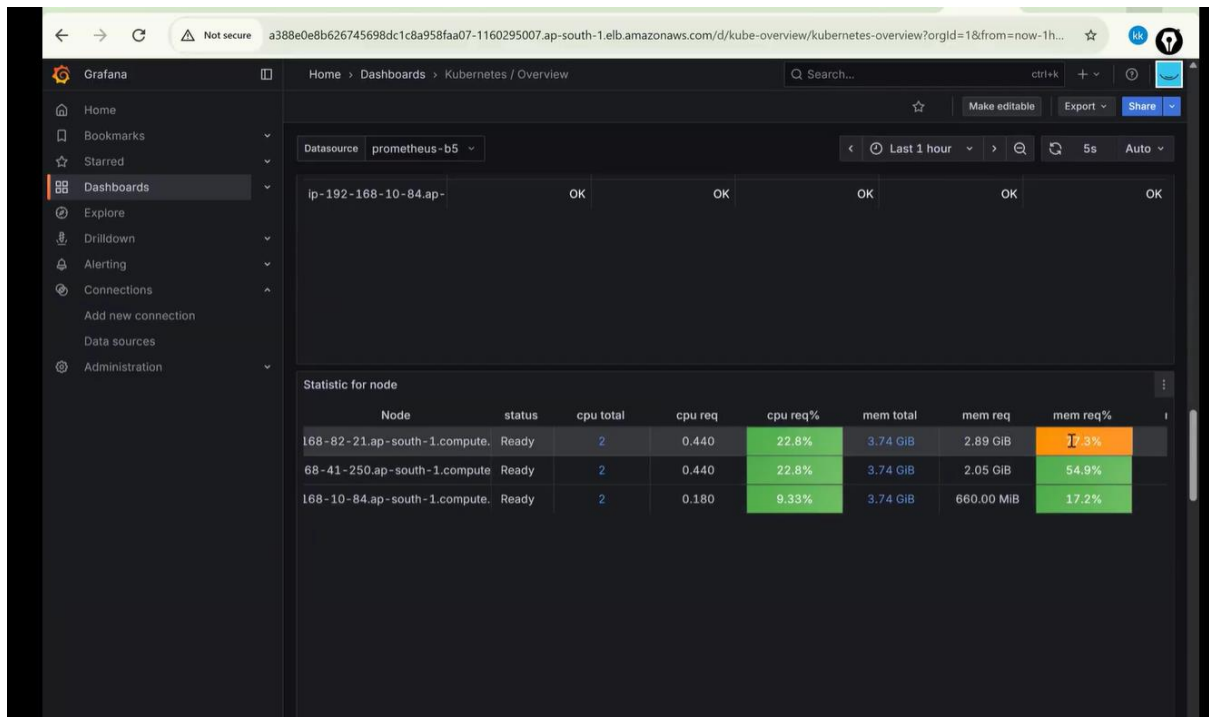


Cluster health:



Autoenable every 15sec-----so it will refresh & update new data in every 15 sec-----





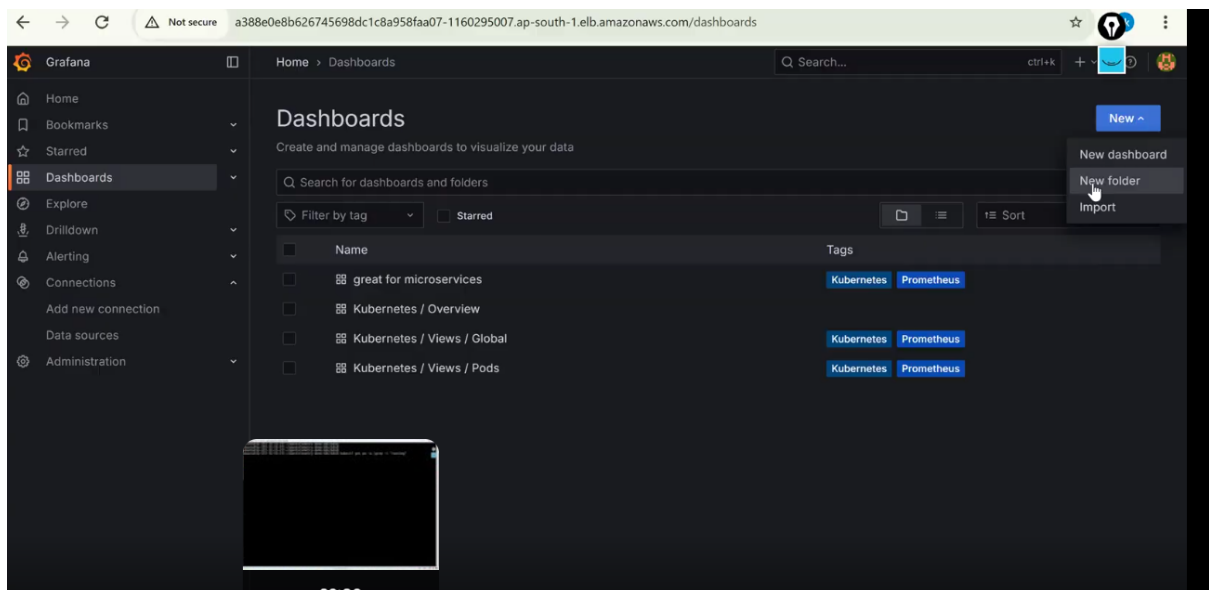
Want to create custom dashboard:

Dashboards creates through Grafana id or import json

We have 3 options-----1.Create new dashboard

2.create new folder-----create dashboards under this folder

3.import



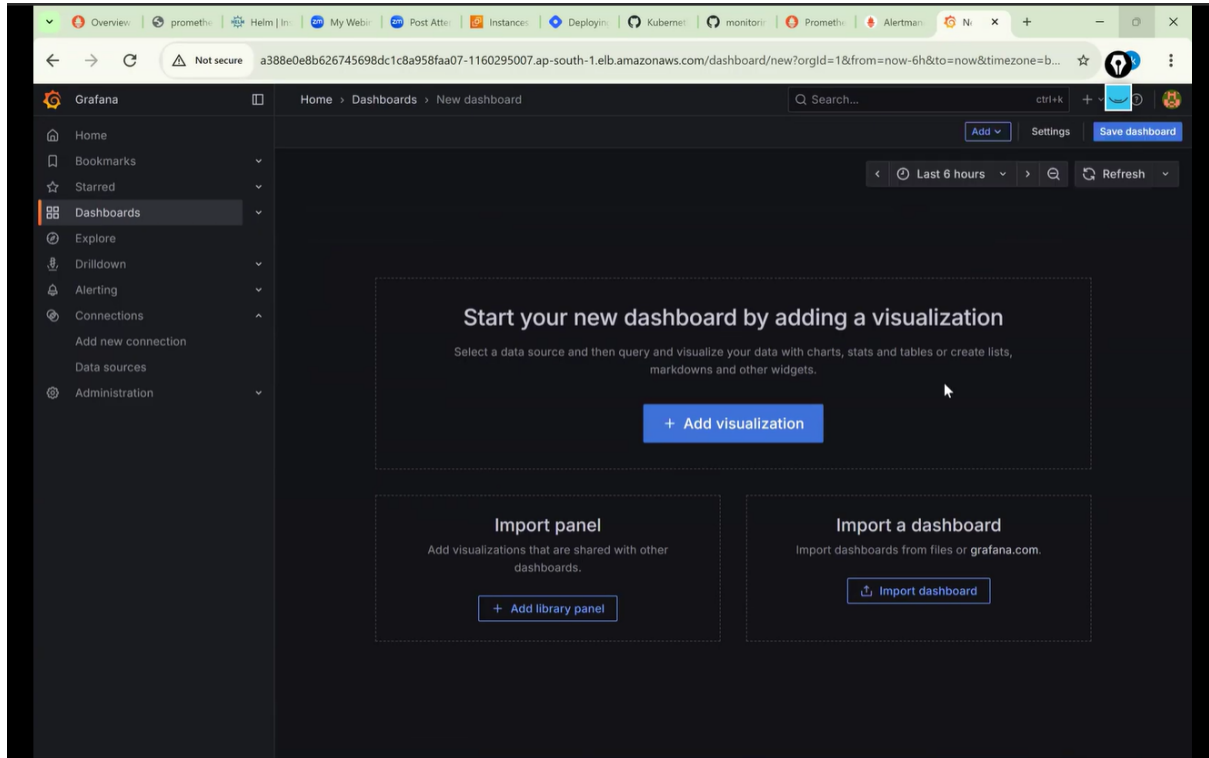
create new folder-----create dashboards under this folder

we use this b/c we can split ----to monitor pods,

to monitor nodes etc

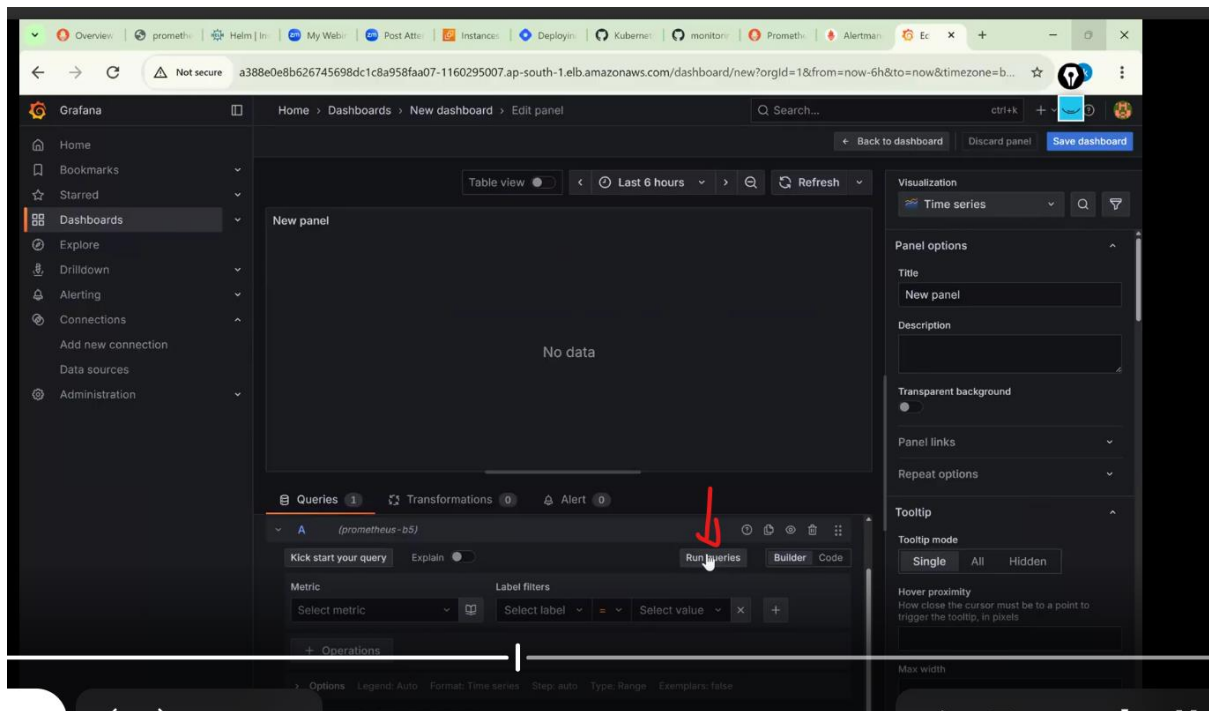
create custom dashboard:

Click-----new dashboard



Click add visualisation:

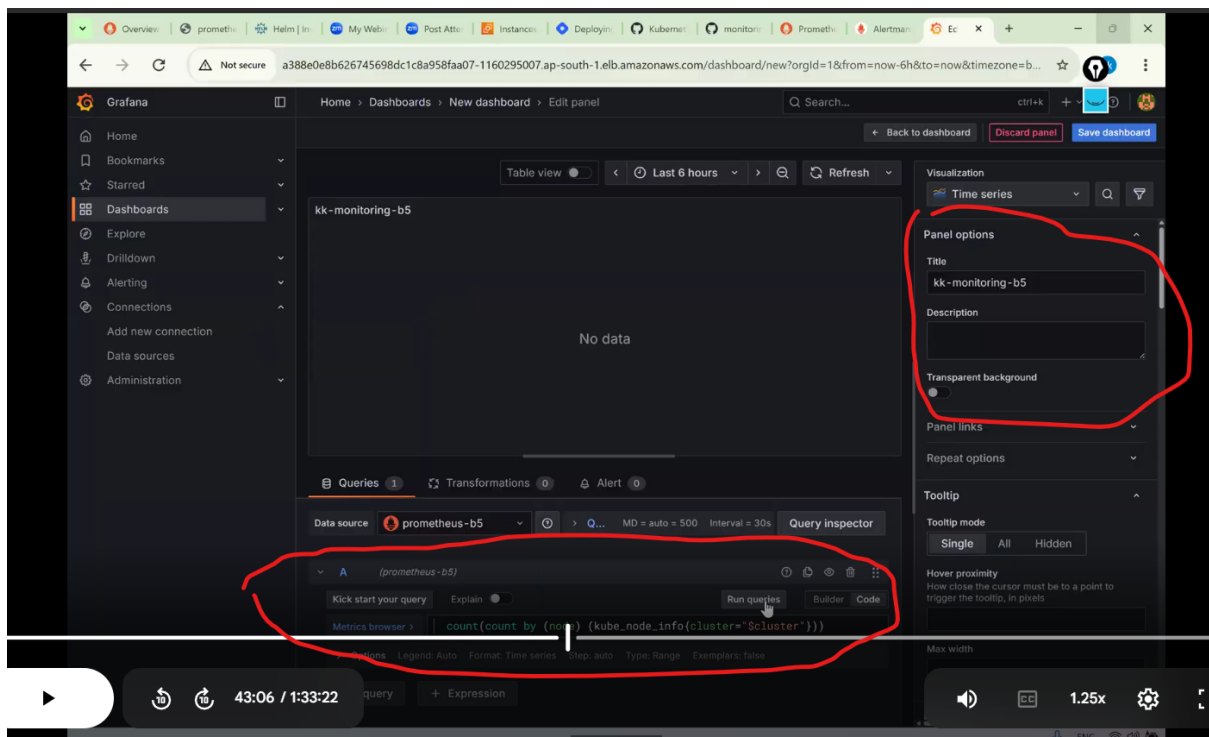
Select data source-----prometheus

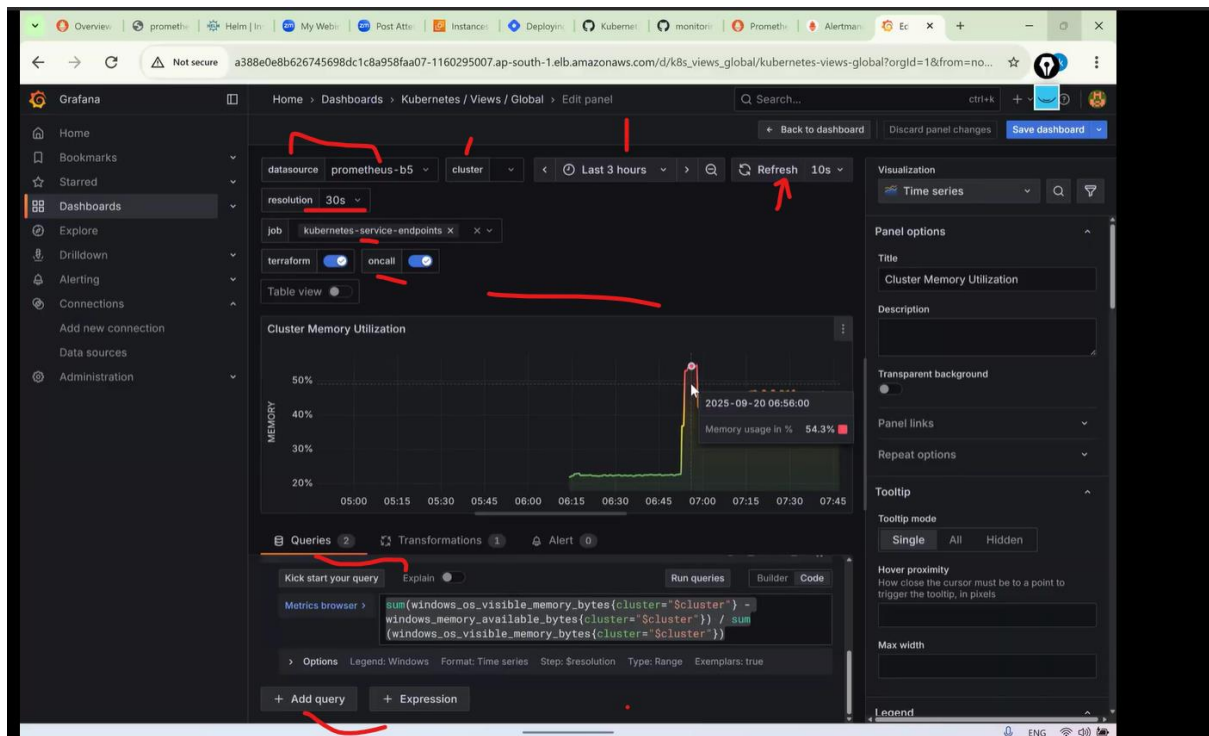


now run the queries from here:

Click on -----code

paste the querie & run queries





What query languages does Grafana support?

🔗 Checks: Integration knowledge

Answer:

Grafana itself doesn't have a query language; it uses the data source's language like PromQL (Prometheus), LogQL (Loki), SQL, Lucene, etc.

How does Grafana work with Prometheus?

🔗 Checks: Monitoring pipeline

Answer:

Prometheus scrapes metrics and stores time-series data. Grafana connects to Prometheus as a data source and uses PromQL queries to visualize metrics.

What is alerting in Grafana?

🔗 Checks: Ops readiness

Answer:

Alerting allows Grafana to evaluate queries periodically and trigger alerts when conditions are met (e.g., CPU > 80%).

6. What alert states exist in Grafana?

🔗 Checks: Alert lifecycle

Answer:

- OK
- Pending

- Alerting
- NoData
- Error

What are recording rules and why do they matter for Grafana?

🔖 Checks: Performance optimization

Answer:

Recording rules pre-compute expensive PromQL queries so Grafana dashboards load faster and reduce Prometheus load.

16. What metrics do you usually visualize for Kubernetes in Grafana?

🔖 Checks: Real-world experience

Answer:

- Node CPU / Memory
- Pod restarts
- Namespace resource usage
- API server latency
- etcd latency

17. How do you visualize pod-level metrics in Grafana?

🔖 Checks: PromQL skills

Answer:

Using metrics from kube-state-metrics and cAdvisor via Prometheus, filtered by namespace and pod labels.

18. How do you build a dashboard from scratch in Grafana?

🔖 Checks: Hands-on ability

Answer (Steps):

19. Add data source
 20. Create dashboard
 21. Add variables
 22. Write queries
 23. Choose visualization
 24. Set thresholds
 25. Save & share
-

◆ Scenario-Based (SRE / DevOps)

19. Grafana dashboard is slow. How do you troubleshoot?

🔖 Checks: Problem solving

Answer:

- Optimize PromQL
- Use recording rules
- Reduce panel refresh rate
- Limit time range
- Check Prometheus performance

20. How do you version-control Grafana dashboards?

🔖 Checks: Platform engineering

Answer:

Using JSON dashboards stored in Git and deployed via Terraform, Helm, or Grafana provisioning.

Alerting Rules and Recording Rules — exactly how interviewers and real projects expect them.

● Alerting Rules (alerts.yml)

🔒 High CPU Usage

groups:

- name: node-alerts

rules:

- alert: HighCPUUsage

expr: avg(rate(node_cpu_seconds_total{mode!="idle"}[5m])) by (instance) > 0.8

for: 5m

labels:

severity: warning

annotations:

summary: "High CPU usage on {{ \$labels.instance }}"

description: "CPU usage > 80% for 5 minutes"

🔒 High Memory Usage

- alert: HighMemoryUsage

```
expr: (node_memory_MemTotal_bytes - node_memory_MemAvailable_bytes)
    / node_memory_MemTotal_bytes > 0.85
```

for: 5m

labels:

severity: warning

annotations:

summary: "High memory usage on {{ \$labels.instance }}"

description: "Memory usage > 85%"

Disk Space Low

- alert: DiskSpaceLow

```
expr: node_filesystem_avail_bytes{fstype!~"tmpfs|overlay"}
    / node_filesystem_size_bytes < 0.15
```

for: 10m

labels:

severity: critical

annotations:

summary: "Low disk space on {{ \$labels.instance }}"

description: "Disk free < 15%"

Pod CrashLoop

- alert: PodCrashLooping

```
expr: rate(kube_pod_container_status_restarts_total[5m]) > 0
```

for: 3m

labels:

severity: critical

annotations:

summary: "Pod is restarting frequently"

description: "Pod {{ \$labels.pod }} in namespace {{ \$labels.namespace }} is crashing"

API High Latency (Histogram)

- alert: HighAPILatency

expr: histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m])) > 1

for: 5m

labels:

severity: critical

annotations:

summary: "High API latency"

description: "95th percentile latency > 1s"

📄 Recording Rules (recording_rules.yml)

👉 Used to **precompute metrics** for performance + faster Grafana dashboards.

CPU Usage Precompute

groups:

- name: node-recording

rules:

- record: instance:cpu_usage:rate5m

expr: avg(rate(node_cpu_seconds_total{mode!="idle"}[5m])) by (instance)

Memory Usage Precompute

- record: instance:memory_usage:ratio

expr: (node_memory_MemTotal_bytes - node_memory_MemAvailable_bytes)
/ node_memory_MemTotal_bytes

API Request Rate

- record: service:http_requests:rate5m

expr: sum(rate(http_requests_total[5m])) by (service)

Error Rate

- record: service:http_errors:rate5m

expr: sum(rate(http_requests_total{status=~"5.."}[5m])) by (service)

📁 How to Load Rules in Prometheus

rule_files:

- "alerts/*.yaml"
 - "recording/*.yaml"
-

📄 Interview One-Liners

Use these in interviews:

✅ Alerting rule:

“Defines conditions to trigger incidents.”

✅ Recording rule:

“Precomputes expensive queries to improve performance and reduce Prometheus load.”

✅ Golden line:

“Alerting rules are for humans, recording rules are for systems.”